

Национальный исследовательский университет ИТМО
Факультет программной инженерии и компьютерной техники

Отчёт к практическому заданию №3
по дисциплине «Системное программное обеспечение»

Студент: Горинов Даниил Андреевич
Группа: Р4116

Санкт-Петербург
2025/2026 учебный год

1 Цели

Сформировать линейный код по CFG набора подпрограмм, вывести его в виде мнемонического листинга и подтвердить корректность исполнения с использованием утилиты из каталога `tools`.

2 Задачи

Подготовка к выполнению (сценарий 1):

1. Составить описание виртуальной машины с набором инструкций и моделью памяти по варианту
 - (a) Изучить нотацию для записи определений целевых архитектур
 - (b) Составить описание ВМ в соответствии с вариантом
 - i. Описание набора регистров и банков памяти
 - ii. Описать набор инструкций: для каждой инструкции задать структуру операционного кода, содержащего описание операндов и набор операций, изменяющих состояние ВМ
 - A. Описать инструкции перемещения данных и загрузки констант
 - B. Описать инструкции арифметических и логических операций
 - C. Описать инструкции условной и безусловной передачи управления
 - D. Описать инструкции ввода-вывода с использованием скрытого регистра в качестве порта ввода-вывода
 - iii. Описать набор мнемоник, соответствующих инструкциям ВМ
 - (c) Подготовить скрипт для запуска ассемблированного листинга с использованием описания ВМ
 - i. Написать тестовый листинг с использованием подготовленных мнемоник инструкций
 - ii. Задействовать транслятор листинга в бинарный модуль по описанию ВМ
 - iii. Запустить полученный бинарный модуль на исполнение и получить результат работы
 - iv. Убедиться в корректности функционирования всех инструкций ВМ

Порядок выполнения:

1. Описать структуры данных, необходимые для представления информации об элементах образа программы (последовательностях инструкций и данных), расположенных в памяти
 - (a) Для каждой инструкции — имя мнемоники и набор операндов
 - (b) Для элемента данных — соответствующее литеральное значение или размер экземпляра типа данных в байтах
2. Реализовать модуль, формирующий образ программы в линейном коде для данного набора подпрограмм

- (a) Программный интерфейс модуля принимает на вход структуру данных, содержащую графы потока управления и информацию о локальных переменных и сигнатурах (задание 2)
 - (b) В результате работы порождается структура данных с описанием образа программы в памяти: набор именованных элементов данных и набор именованных фрагментов линейного кода
 - (c) Для каждой подпрограммы посредством обхода узлов CFG в порядке топологической сортировки сформировать группы инструкций, включая пролог и эпилог
 - (d) Для каждого базового блока сформировать группу инструкций, соответствующих операциям дерева операций
 - (e) Использовать имена групп инструкций для формирования переходов между блоками
3. Доработать тестовую программу, разработанную в задании 2
- (a) Добавить поддержку аргумента командной строки для имени выходного файла, вывод графов CFG сделать опциональным
 - (b) Использовать модуль из п. 2 для формирования образа программы на основе CFG
 - (c) Вывести ассемблерный листинг в выходной файл
 - (d) Проверить корректность решения посредством сборки листинга и запуска бинарного модуля на утилите из `tools`
4. Результаты тестирования представить в виде отчета, в который включить:
- (a) В части 3 привести описание разработанных структур данных
 - (b) В части 4 описать программный интерфейс и особенности реализации разработанного модуля
 - (c) В части 5 привести примеры исходных текстов, соответствующие ассемблерные листинги и примеры вывода запущенных тестовых программ

3 Описание работы

Принятые условия варианта

Типизация — статическая. Формат инструкций — стековый. Банки памяти: `code`, `constants`, `data`, `stack`. Для статической типизации в синтаксической модели обязательно присутствуют `typeRef`, поэтому типы аргументов и возвращаемых значений фиксируются в AST и используются при кодогенерации.

Описание ВМ и мнемоник

ВМ описана в терминах регистров `IP`, `SP`, `BP` и скрытого регистра `IO`. Набор инструкций включает:

- перемещение и загрузку констант:

PUSH_CONST PUSH_ADDR LOAD STORE LOAD_IND STORE_IND

- арифметику и логику:

ADD SUB MUL DIV REM NEG LT LE GT GE EQ NE AND_OP OR_OP

- управление:

JMP JZ CALL ENTER LEAVE RET/RETF

- ввод-вывод:

SET_PORT IN OUT

Описание мнемоник и форматов хранится в `vm/spec.json`.

Структуры данных линейного кода

- `LC_Program`: массивы констант, данных и блоков кода.
- `LC_DataItem`: имя, тип (`const/var`), литерал или размер в байтах.
- `LC_Instruction`: мнемоника и операнды.
- `LC_CodeBlock`: именованный блок инструкций.

Интерфейс и запуск

Основные функции:

```
LC_Program lc_generate_program(const CFG_Analysis* analysis);
int lc_write_assembly(const LC_Program* program, const char* filename);
```

CLI-утилита:

```
./app -o <output.asm> <input_files...>
./app --cfg --cfg-dir <dir> -o <output.asm> <input_files...>
```

Проверка листингов выполнялась утилитой из `tools`:

```
tools/run.bat <asm> <input> exec <out.ptptb> <out.stdout.txt> <out.trace.txt>
```

4 Аспекты реализации

Генерация кода по CFG

Для каждой подпрограммы собирается таблица переменных и параметров. Параметры извлекаются из сигнатур (`typeRef`), размер вычисляется по типу (1 байт для `char/bool`, 4 байта для целых и ссылок). Для массивов размер определяется по типу `array[ N ]` или по максимальному индексу в операциях обращения; при неизвестных индексах закладывается запас 64 элемента.

CFG обходится в порядке, полученном топологической сортировкой на графе SCC, что гарантирует корректную обработку циклов. Для блока входа формируется пролог: сохранение аргументов в локальные переменные (`STORE`) и выделение локального кадра (`ENTER`). Для блока выхода формируется эпилог (`LEAVE`) и возврат `RET` или `RETF`.

Генерация инструкций из дерева операций

IR-узлы преобразуются в стековые инструкции:

- `CONST` → `PUSH_CONST` с помещением значения в пул констант.
- `LOAD/STORE` → `LOAD` или `STORE` по имени переменной.
- Индексация → `PUSH_ADDR`, `INDEX`, `LOAD_IND/STORE_IND`.
- Вызов функции → вычисление аргументов, `CALL` с числом аргументов.

Для булевых и числовых литералов выбирается тип константы, строки и символы размещаются в секции `const_pool`.

Формирование переходов между блоками

Для условных блоков используется `JZ` на ветку `false` и `JMP` на ветку `true`. Для линейного потока формируется безусловный переход на следующий блок, при необходимости лишнее значение на стеке очищается `POP`.

5 Результаты

- Реализован генератор линейного кода и вывод листинга.
- Подготовлено описание ВМ и мнемоник в `vm/spec.json`.
- Проведены тесты с использованием утилит из `texttttools`:

```
tools/run.bat
tools/Portable.RemoteTasks.Manager.exe
```

`tests/vm_instructions.asm`

- Проверка набора инструкций.
- Вывод: ОК (`results/vm_instructions.stdout.txt`).

`tests/calculator.txt` и `tests/calculator.asm`

- Проверка арифметики и ввода-вывода.
- Вывод: 4. Файлы:

```
results/calculator.stdout.txt
results/calculator_asm.stdout.txt
```

`tests/array_demo.txt`

- Проверка индексации массивов.
- Вывод: байты 0x05 0x04 (`results/array_demo.stdout.txt`).

`tests/spo3_demo.txt` и `tests/input.txt`

- Программы без ввода-вывода, вывод пустой.

- Файлы:

`results/spo3_demo.stdout.txt`

`results/input.stdout.txt`

`tests/control_flow_big.txt`

- Стресс-тест управляющих конструкций.
- В `results/control_flow_big.stdout.txt` зафиксировано исключение утилиты, связанное с обращением к неинициализированной памяти.

6 Выводы

Реализовано формирование линейного кода на основе CFG с учётом статической типизации и стековой модели исполнения. Созданы структуры данных образа программы, обеспечена генерация пролога/эпилога, переходов и операций над массивами. Тестирование через утилиту из `tools` подтвердило корректность основных сценариев и выявило ожидаемые ошибки исполнения на сложных входах, что соответствует требованиям задания.